

Collaborating on Neon Rift 3D

Game-context slides for Week 8 GitHub collaboration practice

Two students work on different parts, then combine changes into one working game.

- Use the existing Neon Rift 3D project as the practice repository
- Keep code changes small so students focus on collaboration
- Connect GitHub steps to realistic game-development tasks

Student A

**Enemy feature
StrikerEnemy3D.cs**

Student B

**Player feature
Player3D.cs**

One shared Neon Rift 3D project

Why use Neon Rift for GitHub practice?

The game gives students a concrete reason to use version control.

Shared project

**Different
responsibilities**

Visible results

Need to combine work

- Students can see why teamwork needs structure.
- A small code edit produces an immediate gameplay change.
- GitHub becomes a tool for coordinating real project work, not just submitting files.
- The same workflow can later support their group game projects.

**GitHub is easier to
understand when students
can see what each
teammate is changing in the
game.**

Divide work by responsibility

Before using branches, students should know who owns which part of the game.



Good task division reduces accidental overlap and makes review easier.

Example two-person development task

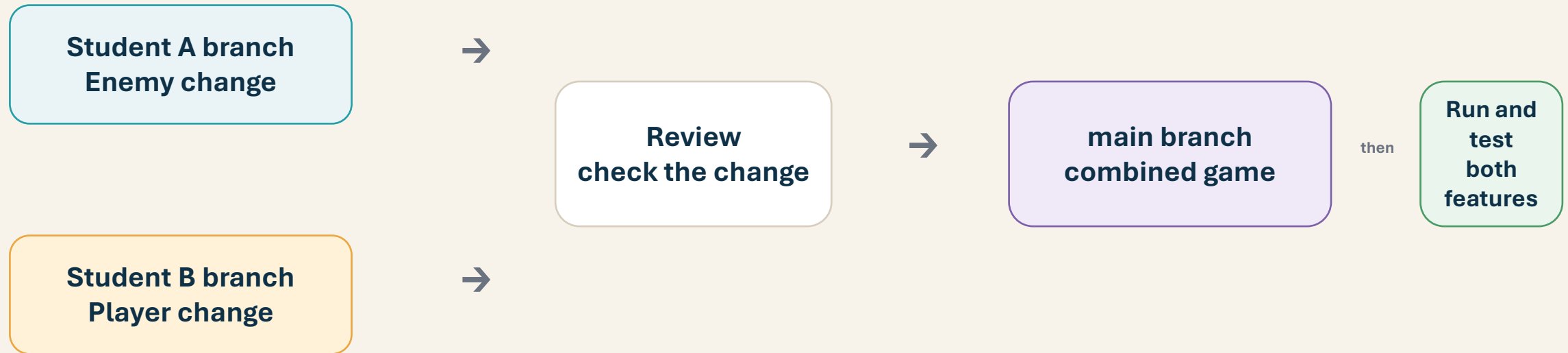
These are intentionally small changes. The lesson focus is GitHub collaboration.

Member	Feature	Main file	Visible result
Student A	Make Striker shoot more often	Scripts3D/StrikerEnemy3D.cs	Striker becomes more challenging
Student B	Increase player movement speed	Scripts3D/Player3D.cs	Player moves faster

- Each task touches a different file first
- Each change is easy to test
- The game result is visible
- GitHub remains the main learning focus

From separate work to one game

GitHub helps both students combine their separate changes safely.



Game-context role
Students should understand what changed in the game before they learn the GitHub commands for combining those changes.

When both students edit the same place

After a normal merge, use a simple README line to demonstrate a conflict.

Original README line
Game objective: Defeat 10 enemies.

Student A changes it to
Game objective: Defeat 15 enemies.

Student B changes it to
Game objective: Defeat 20 enemies.



Merge conflict
Git cannot automatically choose between two different edits to the same line.

Use README.md first, not a complex Godot scene file.

Project ideas students can build from

These are simple starting points for two-person group projects.

Survival Mode

Survive for 60 seconds while enemies keep spawning.

Speed Challenge

Defeat 10 enemies as quickly as possible.

Heavy Enemy Mode

Slow enemies with more health and damage.

Ranged Arena

More Strikers, more dodging, more projectiles.

How to split work

**One member can work on enemy/spawn rules.
The other can work on player, UI, timer, or balancing.**

Plan before coding

Before starting the group project, each pair should make a small work plan.

- 1 What is our game idea?
- 2 What will Member A build?
- 3 What will Member B build?
- 4 Which files will each member likely edit?
- 5 Where could overlap or conflict happen?
- 6 How will we test the final combined game?

**Use the planning sheet
Then begin the GitHub
workflow.**

GROUP ASSIGNMENT

Submission Requirements

Submit one package per group through the assignment submission link.

1 What to submit

- **Completed plan documentation**
Complete and submit the required project documentation.
- **GitHub repository link**
Submit the URL of your group repository.

2 Your GitHub repository should show

- ✓ The final improved code
- ✓ How many branches your group created
- ✓ How the branches were merged
- ✓ Clear commit / merge history showing the development process

ONE GROUP = ONE SUBMISSION • Submit through the assignment submission link